Activity 7

Digital Forensics

6733172621 Patthadon Phengpinij
Collaborators. Claude (for LaTeX styling and grammar checking)

1 Overview

Assuming that you are now a digital forensic trainee, we will learn a few tricks on how to gather information. In this exercise, we will assume that you are using Linux (Debian). There are similar and equivalent tools available on other systems (eg. Windows, Mac OS X) as well.

2 Exercise

Problem 1. Where is your home?

Assuming your friend has sent you a picture of him taken at his home. You want to find out the location of his home by extracting the EXIF information.

Here is the sample file.

https://drive.google.com/file/d/14H93ZHJ9WEU4e-r003MXCxflbvphnAHc/view?usp=sharing

If `imagemagick` is not install, try

```
$ sudo apt install imagemagick
$
```

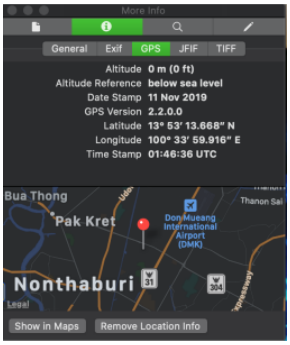
Use 'identify' command (from `imagemagick`) to extract the EXIF information.

```
$ identify -verbose [img_file.jpg]
# Or
$ identify -format '%[EXIF:*]' [img_file.jpg]
```

Find the following information.

- Device that the picture was taken.
- Date and Time that the picture was taken.
- GeoLocation (Lat/Long) of the location that the picture was taken. With proper conversion, you can even place it on google map.

On a modern operating systems (eg. Mac OS X), you can do this directly from the file navigation tool or the image viewer (eg. Preview on Mac OS X).



Solution

Environment: ImageMagick `identify` on Ubuntu 22.04 (WSL2), run on the sample file `IMG_20191111_084637.jpg` downloaded from the link above.

```
$ identify IMG_20191111_084637.jpg
IMG_20191111_084637.jpg JPEG 3648x1680 3648x1680+0+0 8-bit sRGB 6.76379MiB 0.000u 0:00.000
$ identify -format '%[EXIF:*]' IMG_20191111_084637.jpg \
    | grep -E 'Make=|Model=|DateTimeOriginal|GPS(Lat|Long|Date|Time)'
exif:DateTimeOriginal=2019:11:11 08:46:39
exif:GPSDateStamp=2019:11:11
exif:GPSLatitude=13/1, 53/1, 13670654/1000000
exif:GPSLatitudeRef=N
exif:GPSLongitude=100/1, 33/1, 59913940/1000000
exif:GPSLongitudeRef=E
exif:GPSTimeStamp=1/1, 46/1, 36/1
exif:Make=HUAWEI
exif:Model=ELE-L29
```

```
(.venv) nodahttap@LAPTOP-UP5JACSH:~/study/cedt-3-1/CEDT-2110413-Com-Security/activity-07/code$ identify -format '%[EXIF:*]' IMG_20191111_084637.jpg
 | grep -E 'Make=|Model=|DateTimeOriginal|GPS(Lat|Long|Date|Time)'
exif:DateTimeOriginal=2019:11:11 08:46:39
exif:GPSDateStamp=2019:11:11
exif:GPSLatitude=13/1, 53/1, 13670654/1000000
exif:GPSLatitudeRef=N
exif:GPSLongitude=100/1, 33/1, 59913940/1000000
exif:GPSLongitudeRef=E
exif:GPSTimeStamp=1/1, 46/1, 36/1
exif:Make=HUAWEI
exif:Model=ELE-L29
(.venv) nodahttap@LAPTOP-UP5JACSH:~/study/cedt-3-1/CEDT-2110413-Com-Security/activity-07/code$
```

Device. `Make=HUAWEI`, `Model=ELE-L29`, i.e. a **Huawei P30**.

Date and time. `DateTimeOriginal` is **11 November 2019, 08:46:39** (camera local time). The GPS clock stamps the same shot at `01:46:36` UTC, so the local time is UTC+7, which agrees with the location in Thailand.

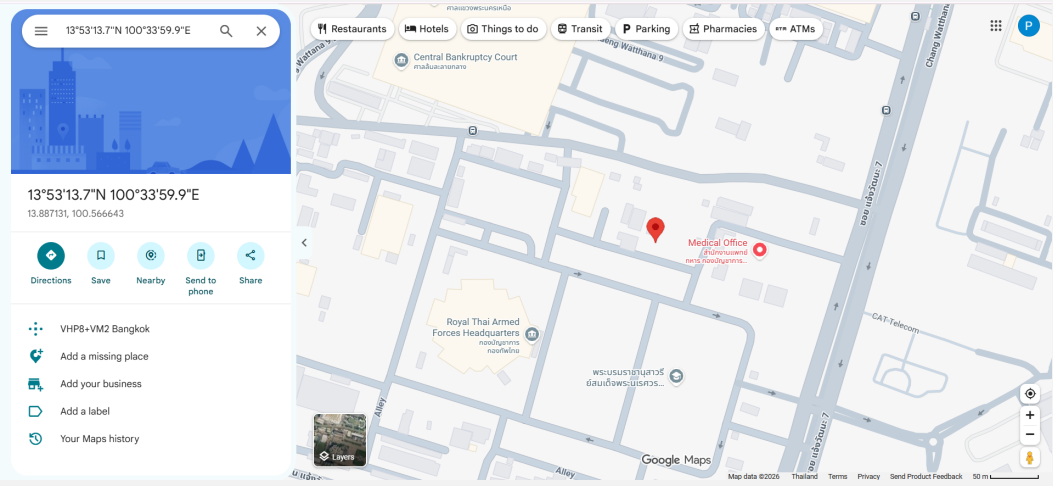
GeoLocation. EXIF stores each coordinate as three rationals (degrees, minutes, seconds), with the hemisphere in the `Ref` tag. Converting with decimal $= d + \frac{m}{60} + \frac{s}{3600}$:

$$\text{Latitude} = 13 + \frac{53}{60} + \frac{13.670654}{3600} \approx \mathbf{13.887131^\circ\ N} \quad (13^\circ 53' 13.67'' \text{N})$$

$$\text{Longitude} = 100 + \frac{33}{60} + \frac{59.913940}{3600} \approx \mathbf{100.566643^\circ\ E} \quad (100^\circ 33' 59.91'' \text{E})$$

Both references are positive (`N`, `E`), so the point is `13.887131, 100.566643`, in northern Bangkok, Thailand:

https://www.google.com/maps?q=13.887131,100.566643



Problem 2. Find a deleted file.

You are presented with a USB drive that may contain a deleted evidence file. Please recover the secret file on the disk.

Note:

You may use the disk image file[1] that I have prepared and skip the preparation and the first two steps.

[1] https://drive.google.com/file/d/1iTKoE9M9rxFa6rdh7I1Qh27AHgSpMwDk/view?usp=sharing

Preparation:

Create a sample text file on your usb drive (eg. `secret.txt`). Then delete it.

Steps:

1. Plug in the USB drive to a computer (Linux). Use the lsblk command to identify the device and its partition number. You should see a device name and partition number (e.g. `/dev/sdc2`).

```
$ lsblk
$
```

2. Use the dd command to clone the device and an (disk) image file for offline forensic.

```
$ dd if=/dev/sdc2 of=disk.img bs=4M status=progress
$
```

3. Install the testdisk tool[2]. On debian base distribution just do

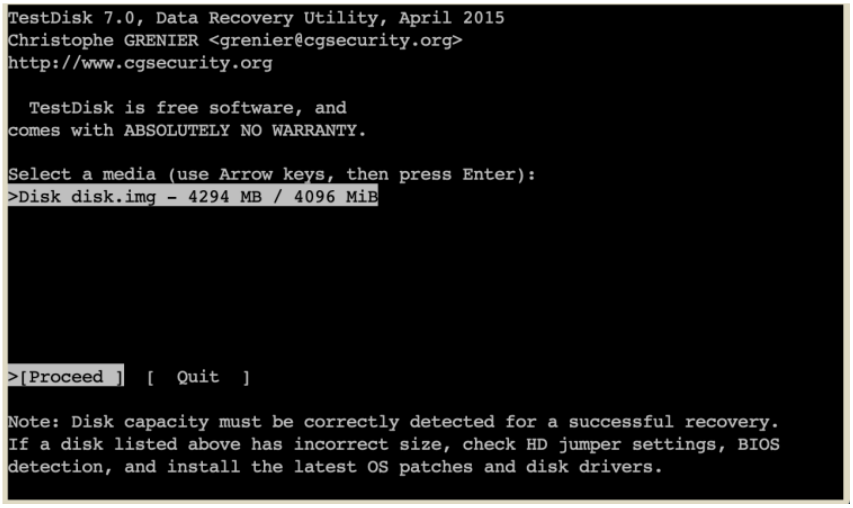
```
$ apt install testdisk
$
```

[2] You may download it from https://www.cgsecurity.org/wiki/TestDisk_Download.

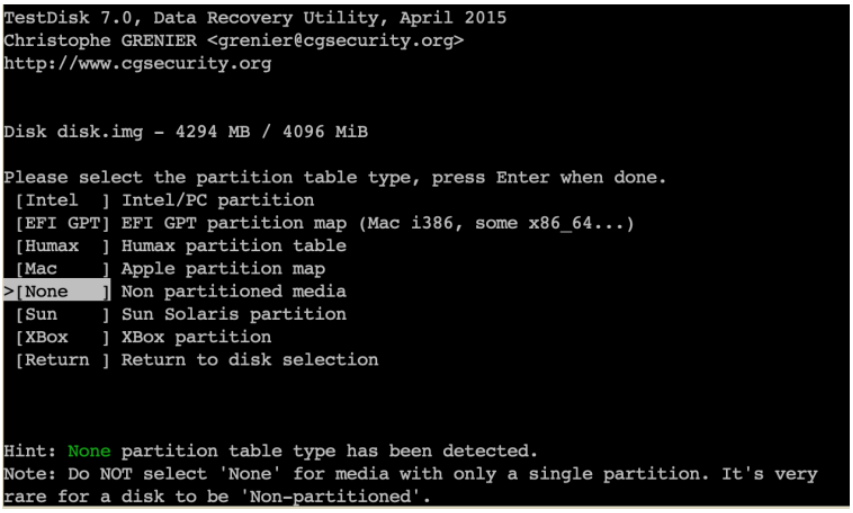
4. Make a loop device to the `disk.img` file. Then launch

```
$ testdisk disk.img
$
```

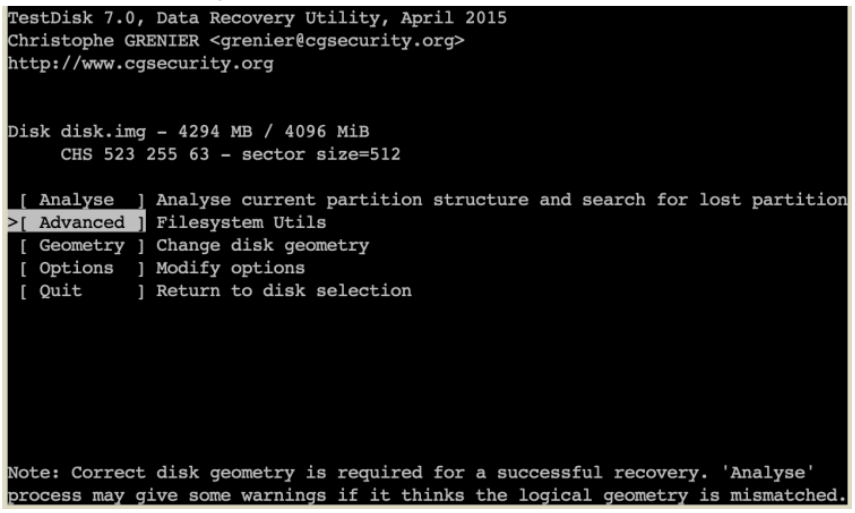
5. Navigate to the filesystem and recover the deleted file.



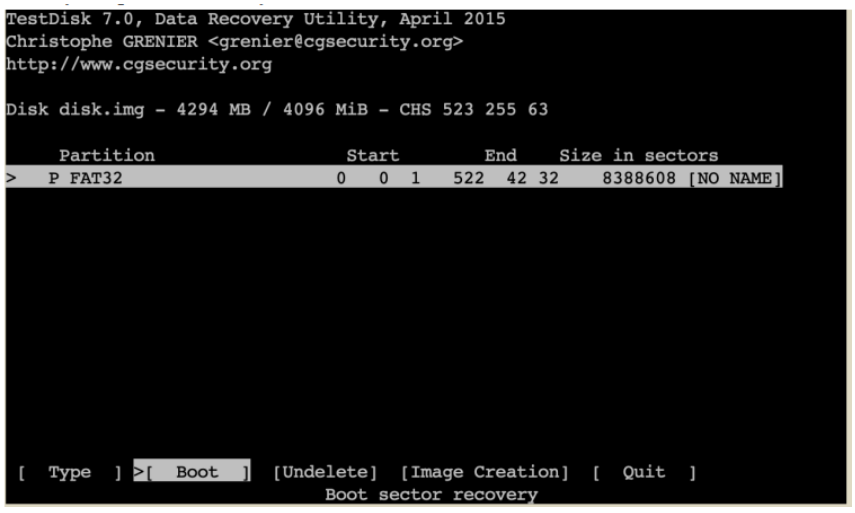
Since we dump only the partition, there is no partition map (None).



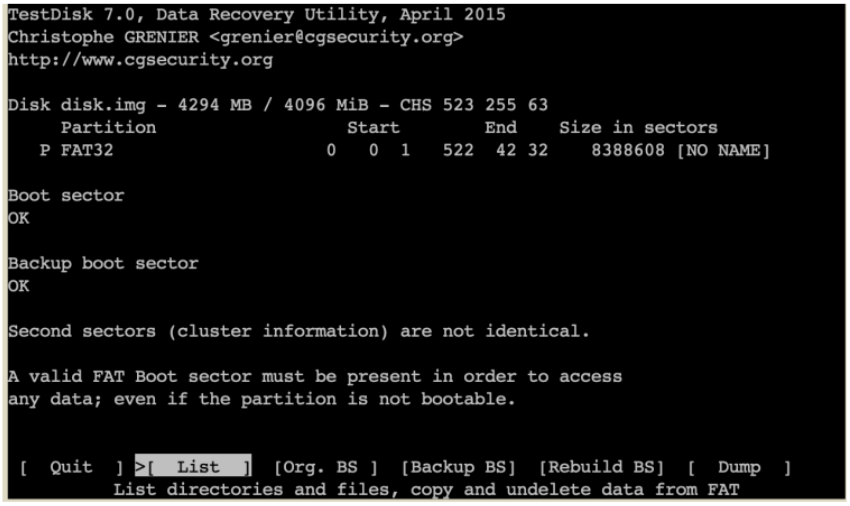
Select Advanced Filesystem Utils.



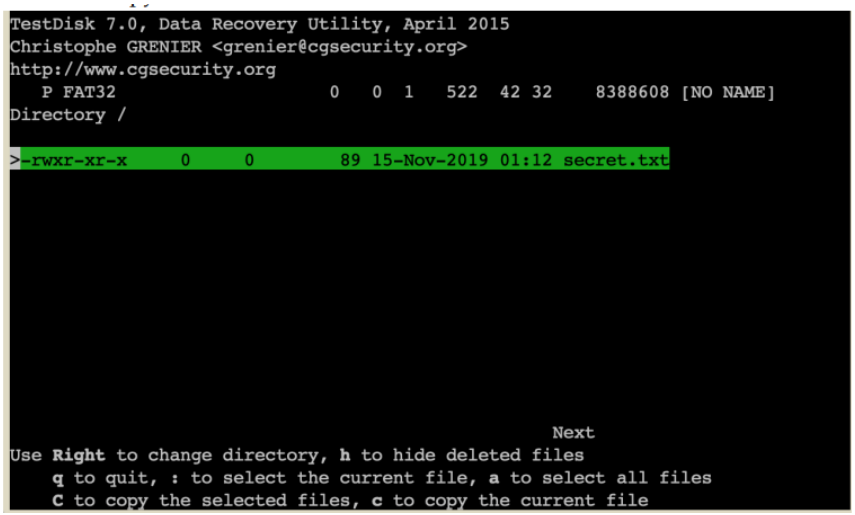
Select your partition/filesystem.



List all files.



Select and copy the deleted file out.



Note that the TestDisk tool supports several file systems and partition map schemes. You may want to experiment with your own system.

Solution

Environment: TestDisk 7.1 on Ubuntu 22.04 (WSL2). No USB drive was available, so, as the note allows, I used the prepared disk image and skipped the preparation and steps 1–2 (`lsblk` / `dd`). Step 4's loop device is not needed either: TestDisk opens the image file directly.

Inspect the image. The zip holds a single 4 GiB `disk.img`, which is a bare FAT32 filesystem (a partition dump with no partition table):

```
$ unzip disk.img.zip
$ file -s disk.img
disk.img: DOS/MBR boot sector, code offset 0x58+2, OEM-ID "mkfs.fat", sectors/cluster 8,
Media descriptor 0xf8, sectors/track 63, heads 255, sectors 8388608 (volumes > 32 MB),
FAT (32 bit), sectors/FAT 8184, serial number 0xb3894437, unlabeled
```

Recover with TestDisk. `testdisk disk.img` → `[Proceed]` → partition table type `[None]` (TestDisk itself hints *"None partition table type has been detected"*). For a non-partitioned medium it goes straight to the filesystem menu, where `[Undelete]` (*File undelete*) is the same tool as *Advanced* →

Undelete in the handout's screenshots:

```
Disk disk.img - 4294 MB / 4096 MiB - CHS 523 255 63

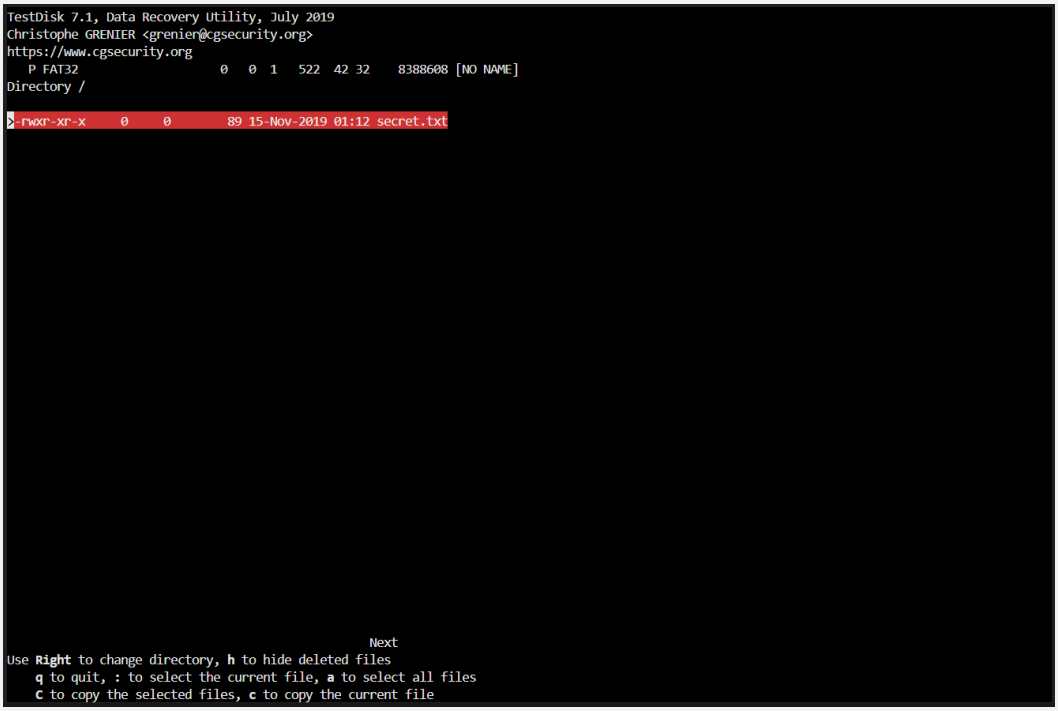
     Partition                  Start        End    Size in sectors
>   P FAT32                    0   0  1   522  42 32    8388608 [NO NAME]

 [  Type  ]  [  Boot  ] >[Undelete]  [Image Creation]  [  Quit  ]
                              File undelete
```

TestDisk lists the deleted entries of `/` (shown in red in the UI); the only one with data is `secret.txt` (89 bytes, last written 15-Nov-2019 01:12). Pressing `c` on it and then `C` to accept the current directory as the destination copies it out:

```
   P FAT32                    0   0  1   522  42 32    8388608 [NO NAME]
Directory /secret.txt
Copy done! 1 ok, 0 failed
>-rwxr-xr-x     0     0        89 15-Nov-2019 01:12 secret.txt

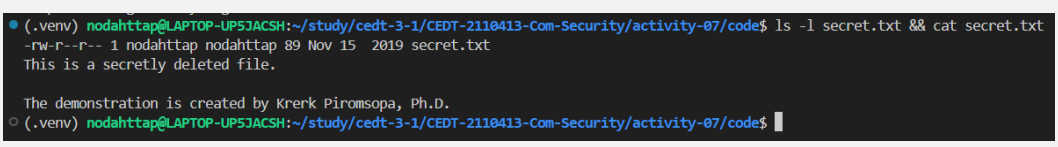
Use Right to change directory, h to hide deleted files
    q to quit, : to select the current file, a to select all files
    C to copy the selected files, c to copy the current file
```



The recovered secret file:

```
$ cat secret.txt
This is a secretly deleted file.

The demonstration is created by Krerk Piromsopa, Ph.D.
$
```



Cross-check with The Sleuth Kit. `fls` marks deleted entries with `*`. Besides `secret.txt` it also finds the leftovers of the editor used to write it (Vim's `.secret.txt.swp` and its 8.3 short name `_ECRET~1.SWX`). `icat -r` recovers the same 89 bytes from inode 7:

```
$ fls -r -d disk.img
r/r * 5:  .secret.txt.swp
r/r * 7:  secret.txt
r/r * 8:  _ECRET~1.SWX
$ icat -r disk.img 7
This is a secretly deleted file.

The demonstration is created by Krerk Piromsopa, Ph.D.
```

Why it works. Deleting a file on FAT only marks its directory entry as free (the first byte of the name becomes `0xE5`) and releases its clusters in the FAT; the data itself stays on disk until something overwrites it. The entry still holds the starting cluster and the file size, so a tool can read the data straight back.

Enjoy!